Smart Parking Lot Simulation Study

How many kilometres of driving does the app save, compared with finding a spot on your own?

Ido Tal · Aviv Elimelech · Harry Cohen · October 2026

Summary

11.7% less driving

to find a parking spot, for every car entering our 32-spot lot, compared with a methodical driver who has no app. 95% confidence interval 11.6% to 11.8% · 5,000 simulated days · 838,792 simulated cars · seed 2026

31.12 m → 27.49 m
average drive from the entrance to the spot, without the app → with the app

0.61 km per day
driving removed in our lot (about 168 cars a day), 95% CI 0.60 to 0.62

56%
less driving when the other drivers just follow the lanes and take the first free spot they see

21% · 33%
less driving in a 128-spot and a 288-spot lot built from the same pattern

57 km per day
driving removed in the 288-spot lot (1558 cars a day)

36%
less driving when the lot is almost full (97%+ taken)

What we did

We built a simulation of a parking lot with the same layout and the same routing code as our app, and ran 5,000 busy weekdays through it twice. On the first pass every driver finds a spot the way people do without help: looking down the aisle and heading for the nearest part of the lot they have not looked at yet. On the second pass the app tells every driver which spot is free (from the sensors) and sends them along the shortest path to the nearest one, using Dijkstra's algorithm. Both passes see exactly the same cars, arriving at the same times and staying the same lengths of time, so the only thing that differs is how the spot is found.

What we found

- The app never makes a trip longer, because its route is the shortest possible. What matters is how much shorter, and that depends on how full the lot is and how big it is.
- In our small prototype lot the saving is 11.7%, or about 3.6 m per car. The lot is small enough that a driver can see most of it from the entrance, which limits what help is possible.
- The saving grows with the size of the lot, from 12% at 32 spots to 33% at 288 spots, because the larger the lot, the less a driver can see and the longer an unaided search takes.
- It grows with how full the lot is, with how long cars stay, and with the share of drivers who use the app.

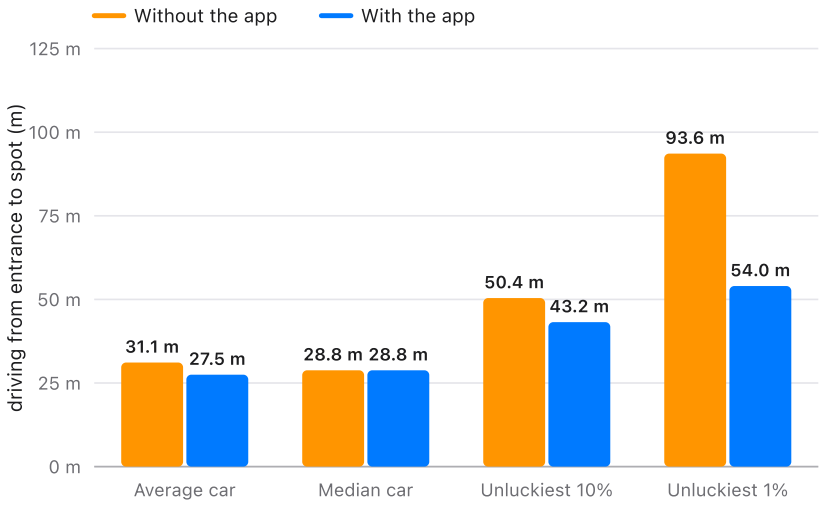


Figure 1. Distance from the entrance to the parking spot, for the average car, the median car and the unluckiest 10% and 1% of cars. 5,000 simulated days.

1. Why this matters

Finding a parking spot is wasted driving. Every metre between the entrance and the spot you finally take is a metre that gets you nowhere: it costs time and fuel, produces emissions, and in a busy lot it adds cars to the aisles, which slows everyone else down and makes the next search longer. Researchers who study this call it *cruising for parking* (Shoup, 2006). Drivers cannot avoid it because from inside a car you can see only the spots in front of you, and you cannot know whether the next aisle has room.

The fix is information. If the lot knows which spots are free, it can tell each driver where to go. That needs two things: sensors that report the state of the lot, and a program that turns that state into a route. That is our project.

2. The project in one page

Our parking lot has an ESP32 board with two infrared distance sensors and an LED strip. The entrance sensor (pin D34) notices that a car has arrived. The spot sensor (pin D35) reports whether the monitored spot is taken, and the LED strip on pin D26 shows it to everybody walking by: green when open, red when taken. The board streams its readings over USB to a web app, which cleans them up (a reading must hold for three samples before it counts), decides that a car has arrived, finds the nearest free spot, and draws the route on a live map and on a phone mockup with turn-by-turn directions. The other spots of the demo lot are simulated, so one real spot sits in a full virtual lot.

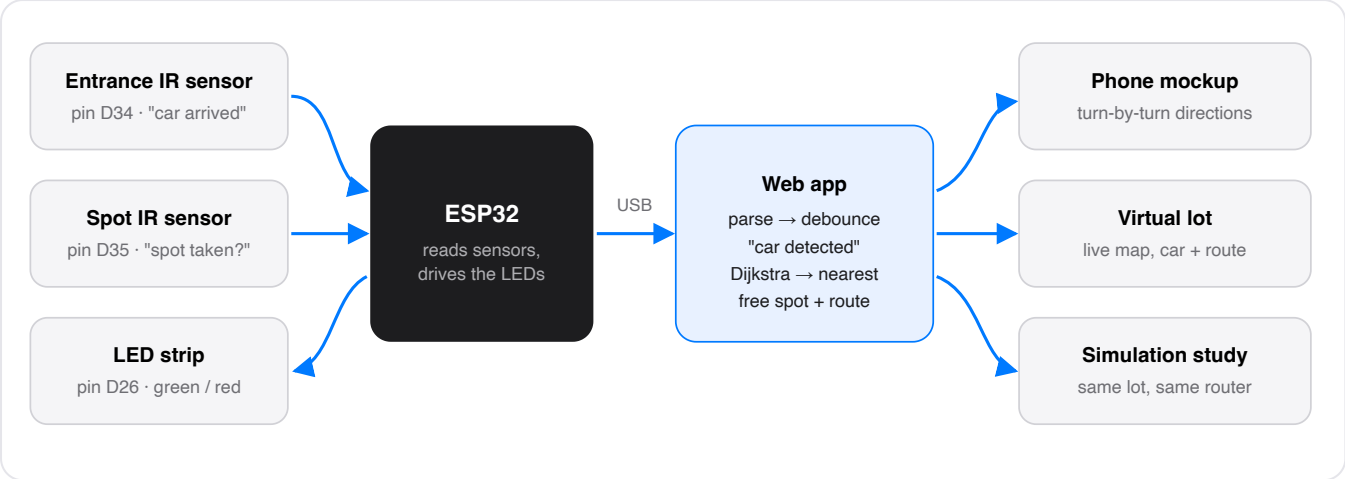


Figure 2. How the pieces connect. The simulation study in this report uses the same lot model and the same routing as the live app.

Why a simulation

The hardware demo proves that the system works for one car and one spot. It cannot tell us how much driving it saves in a real lot, with hundreds of cars a day and spots filling and emptying. A real lot cannot be tested with and without the app on the same day with the same cars, but a simulation can: we replay identical days under two policies and compare. That is the only way to isolate the effect of the app.

What we measure

The main number is **search driving**: the distance a car drives between the entrance and the spot it parks in. We also report the drive back out, the time and the CO_2 that distance represents, and the wasted trips of drivers who arrive when the lot is full.

3. How the app chooses a route

The lot is a grid. Drivable cells (the lanes and the entrance) are the nodes of a graph, and neighbouring lane cells are joined by edges. A spot is reached from a lane cell next to it. When a car is detected, the app needs the free spot with the smallest driving distance from the entrance. That is a shortest-path problem, and Dijkstra's algorithm (1959) solves it exactly:

```
dist[entrance] = 0, every other cell = infinity
repeat: take the unfinished cell with the smallest dist (that distance is now final)
        for each neighbour: if dist[cell] + cost < dist[neighbour], update it
spot distance = (distance of its lane cell) + 1 (the turn into the spot)
route = free spot with the smallest distance, walked back along the cells that set it
```

The figure below shows the result for one lot state: each lane cell is labelled with its shortest distance from the entrance, and the app's chosen spot is outlined. Read it like a map of "how far is each cell from the door". The app takes the free spot whose neighbouring lane cell has the lowest number.

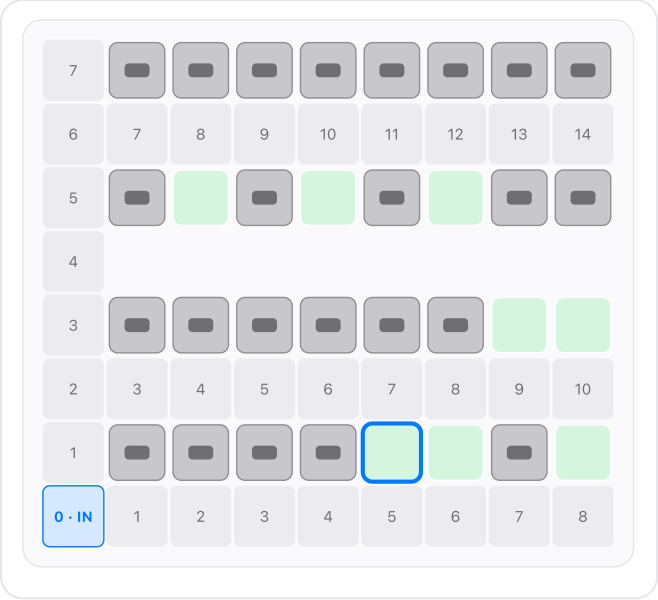


Figure 3. Dijkstra's distances (in cells of 3.6 m) for one example lot with 24 of 32 spots taken (grey cars). The outlined spot is the nearest free one.

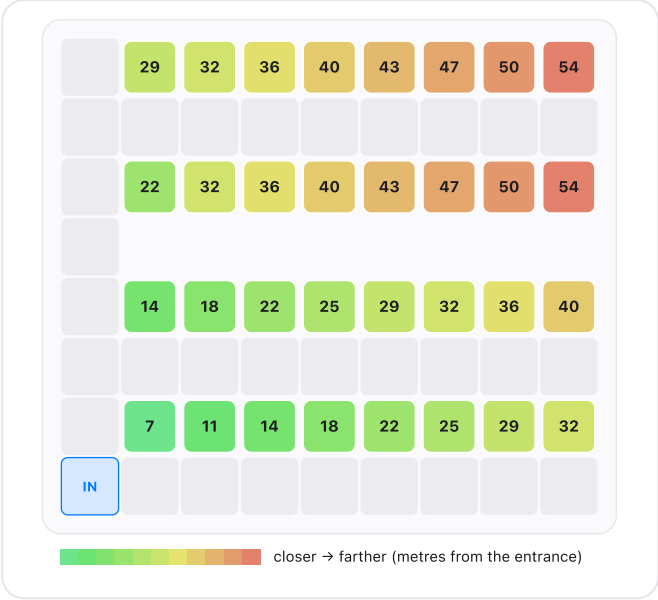


Figure 4. Drive from the entrance to every spot of the empty lot, in metres. The nearest spot is 7.2 m away, the farthest corner 54 m.

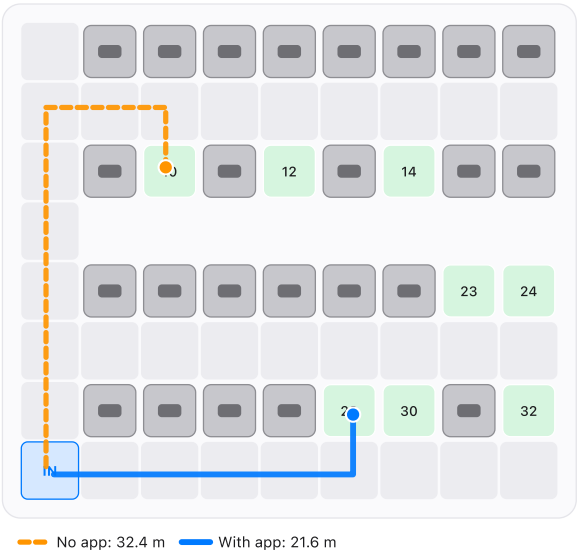


Figure 5. One arrival, two routes. This example (24 of 32 spots taken) was picked because the routes differ; it is an illustration, not the average case, which is 11.7% shorter. The driver without the app (orange, dashed) looks down the aisle, does not see a free spot close by, and ends up farther in; the app (blue) knows which spot is free and goes straight to the nearest one.

Dijkstra and the code that ships in the app

In our lot every step from one lane cell to the next costs the same, one cell. When all costs are equal, Dijkstra's algorithm visits cells in the same order as a breadth-first search, and gives the same distances. The router that runs in the live app (`src/lot/pathfinding.ts`) is that breadth-first search. For the study we wrote a full Dijkstra with a priority queue (`src/sim/dijkstra.ts`), so it keeps working if edges ever get different costs (a one-way aisle, a speed bump), and we check against the shipped router: in 1920 random lot states, from empty to nearly full, both pick the same spot at the same distance every time.

Driver without the app

This is the part the result depends on most, so it is modelled carefully and kept strong. The driver enters at the entrance and looks down each straight lane for up to 14.4 m (4 cells); this is roughly what you can make out when

you turn into an aisle. If any spot in view is free, they take the closest one. If not, they drive to the nearest lane cell with spots they have not yet looked at, look again, and repeat. They never re-check a place they already saw. A person with no app, no memory problems and good judgement would drive about like this, which makes it a demanding comparison for the app. A second, weaker driver is tested too: one who only follows the lanes with random turns and takes the first free spot they see.

> Because the comparison driver is already sensible, the percentages in this report are conservative. Real drivers also hesitate, circle back and give up on aisles that look full.

4. The simulation model

The lot and the scale

The lot has the layout of the live demo: 32 spots in four rows of eight, two aisles, a lane up the left side, and the entrance at the bottom left. One grid cell stands for 3.6 m of driving. That makes the lot 32.4 m by 28.8 m, or 29 m² per space, which is inside the usual 25 to 35 m² per space of a surface lot. The percentages in this report do not depend on this scale (every distance is multiplied by the same factor); the absolute metres and kilometres do, in proportion.

Demand

We simulate a busy weekday from 07:00 to 19:00. In each hour a random number of cars arrives (Poisson, which is the standard model for independent arrivals), at an average rate that peaks at 20 cars per hour in the morning and midday. Arrival times within the hour are uniform. Each car stays for a random time with a mean of 90 minutes (lognormal, so most stays are close to the mean and a few are much longer, as in real lots). A car that arrives when every spot is taken goes away; this happens about 6.1 times a day.

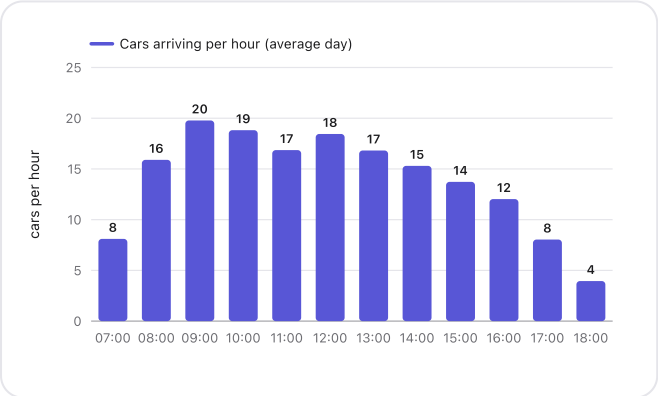


Figure 6. Average cars arriving per hour.

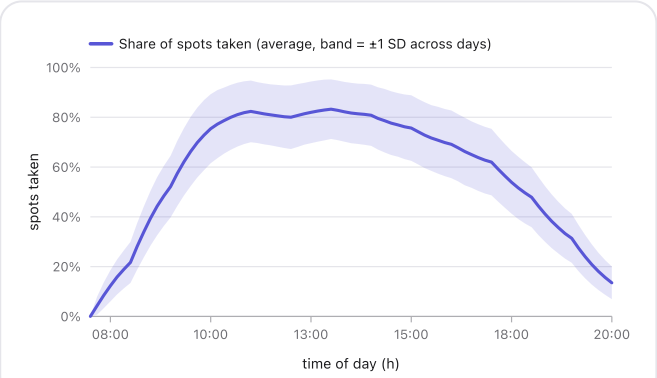


Figure 7. Share of spots taken over the day: peak 83% on average. The band is one standard deviation across days.

The assumptions in one table

ASSUMPTION	VALUE	WHY
Lot	32 spots, 9 × 8 cells	Same as the live demo
Cell size	3.6 m	29 m² per space, typical surface lot
Driver view down an aisle	14.4 m (4 cells)	Six or seven spots; generous to drivers with no app
Arrivals	8, 16, 20, 20, 18, 20, 18, 16, 14, 12, 8, 4 cars/h (07-19)	Busy weekday; peak occupancy near 83%
Time parked	mean 90 min, lognormal σ = 0.7	Work or campus style lot
Full lot	car leaves	No queue modelled
Speed in the lot	10 km/h	Typical 5 to 10 mph limits (used only for time)
Emissions	0.12 kg CO_2 per km	Typical passenger car; stop-and-go is worse
Simulated days	5,000 (headline), 1,000 per sensitivity point	Enough for a very narrow interval

The parameters were fixed from the reasoning above, before the results were looked at. The only thing checked beforehand was that the demand profile gave a plausible occupancy (a peak around 83% on average, a few cars turned away); none of the settings was adjusted to change the size of the saving.

Paired days

For every simulated day, the list of cars (arrival time, time parked, driver) is generated once and then replayed under each policy. The number of occupied spots at any moment depends only on arrival times and stays, so both policies have exactly the same number of cars parked and turned away; only *which* spot each car takes, and how far it drives to it, differs. This removes almost all of the random noise from the comparison and is why the confidence interval is so narrow.

One simplification: a search is treated as instant relative to other cars' arrivals and departures. A search takes under a minute, while cars arrive every few minutes, so the effect is small. It is a limit of the model, not of the app.

5. Method and verification

This is a Monte Carlo simulation: we draw random days from the demand model, run each through both policies, and average. Four things make the number trustworthy.

1. **Paired comparison.** The same cars in both runs, as described above.
2. **Enough days.** The headline uses 5,000 independent days. Figure 8 shows the running estimate: it settles after a few hundred days and does not move afterwards.
3. **An honest interval.** The 95% confidence interval comes from a bootstrap that resamples whole days (days are independent, cars within a day are not): 11.60% to 11.78% around 11.69%.
4. **Self-checks.** The program refuses to run unless these checks pass:

	CHECK	RESULT
PASS	Dijkstra matches the app router	1920 random lot states (0 to 31 spots taken): 0 differences in chosen spot or distance
PASS	Distances match Floyd-Warshall	1024 node pairs compared: 0 differences
PASS	App route is never longer than a search	6400 searches: 0 shorter than the app, 0 ended in a taken spot or no spot
PASS	Cars are conserved and paired	40 days: 6762 arrivals = 6601 parked + 161 blocked, identical in both policies
PASS	Same seed, same numbers	seed 7 twice: 36641/32329 and 36641/32329; seed 8: 37856/33068
PASS	No same-state counterexample in a run	16975 parked cars compared with the app's choice for the same lot state: 0 violations
PASS	Scaled lot generator reproduces the demo lot	72 cells compared: 0 differences, 32 spots

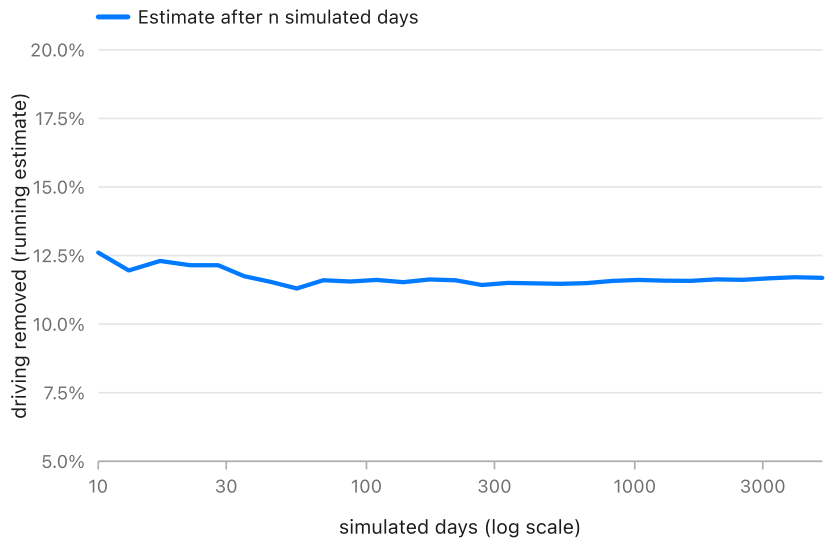


Figure 8. Running estimate of the saving as simulated days are added (log scale).

What the percentage means

The saving is computed from totals: (distance driven without the app – distance with the app) ÷ distance driven without the app, over all cars of all days. It is not an average of per-day percentages (which would over-weight quiet days). One simulated day can differ from the average: across days the daily saving has a standard deviation of 3.0 percentage points.

6. Results: our 32-spot lot

MEASURE	WITHOUT THE APP	WITH THE APP	DIFFERENCE
Average drive to the spot	31.12 m	27.49 m	–3.64 m (–11.7%)
Median car	28.8 m	28.8 m	0.0 m
Unluckiest 10% of cars	50.4 m	43.2 m	–7.2 m
Unluckiest 1% of cars	93.6 m	54.0 m	–39.6 m
Longest single search	97 m	54 m	–43 m
Spread (std. deviation)	16.7 m	11.5 m	
Search driving per day	5.22 km	4.61 km	–0.61 km (CI 0.60 to 0.62)
Search time per car at 10 km/h	11.2 s	9.9 s	–1.3 s
Drive back out, per car	28.4 m	27.5 m	–0.9 m
Search + drive out, per car	59.5 m	55.0 m	–7.6% (CI 7.6% to 7.7%)
Wasted tours of the full lot, per day	0.62 km	0 km	with these counted: –21.0%

The headline is the first row: 11.7% less driving to find a spot. Parking nearer the entrance also shortens the drive out afterwards, so counting both directions the saving is 7.6%. The last row is an extra not included in the headline: when the lot is full, a driver without the app tours the whole lot, finds nothing and drives out; with the app they see "lot full" before turning in. Counting those wasted tours the saving is 21.0%.

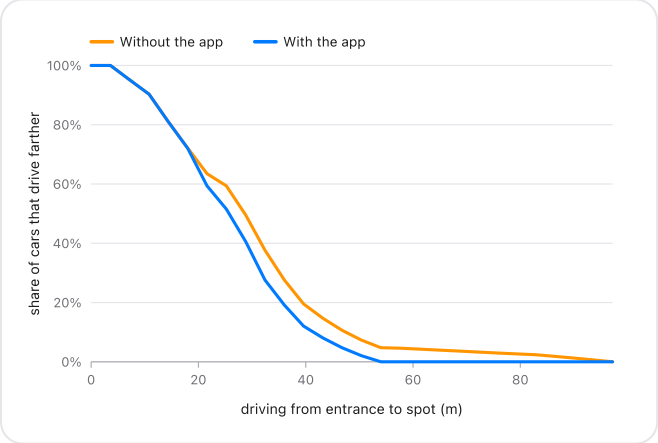


Figure 9. Share of cars that drove farther than a given distance. Without the app 10.6% of cars drive more than 50 m; with the app 4.7%.

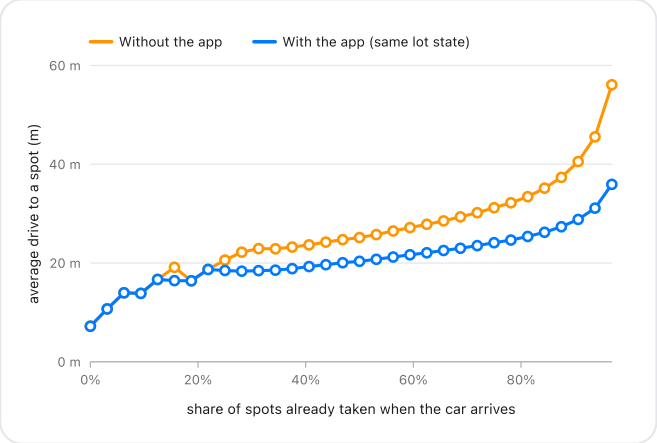


Figure 10. Average drive for cars arriving at the same lot state, by how full the lot already is. At the same state, the app is never longer. The gap opens as the lot fills.

Figure 9 shows where the saving comes from: it is not that every car drives less (the median car drives the same distance either way), it is that the unlucky cars stop driving a long way. When the near spots are free, both policies take them. When they are taken, a driver without the app has to search, and the longer the search the bigger the gap (Figure 10): about 56 m against 36 m when the lot is nearly full.

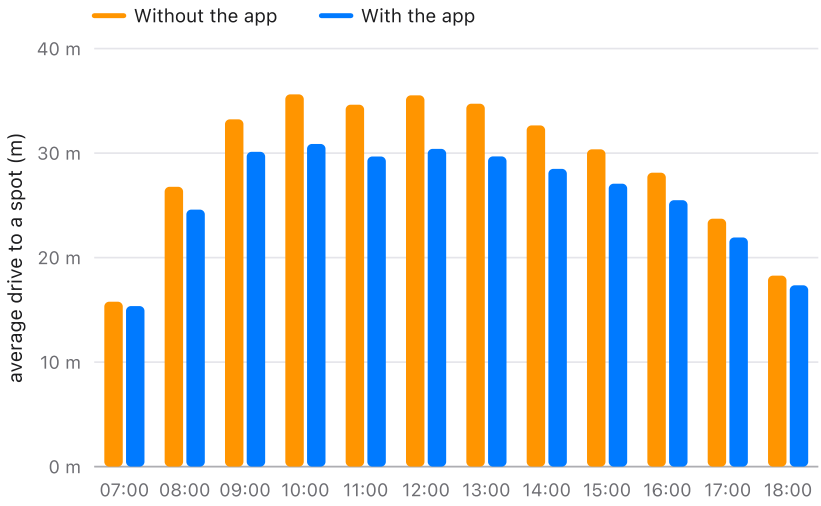


Figure 11. Average drive per car by hour of arrival. The saving follows the rush hours.

7. What would happen in a real lot

Our prototype has 32 spots because it is a demonstration. To estimate what the app does in a real lot, we built larger lots from the same pattern (more columns, more blocks of aisles; the generator reproduces the demo lot exactly at 32 spots, as one of the self-checks shows) and scaled the demand in proportion to the number of spots, so every lot is equally busy.

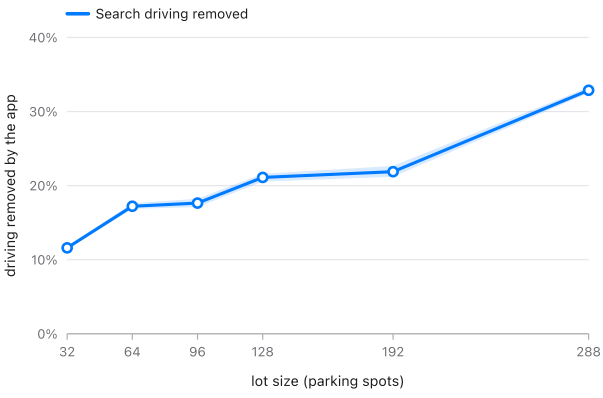


Figure 12. Share of search driving removed, by lot size.

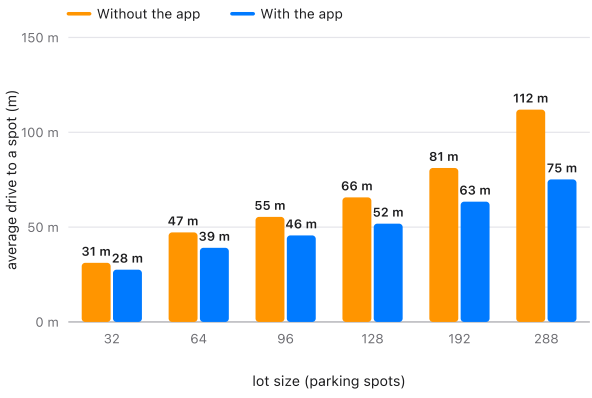


Figure 13. Average drive from the entrance to the spot, by lot size.

SPOTS	CARS PER DAY	WITHOUT THE APP	WITH THE APP	DRIVING REMOVED (95% CI)	SAVED PER DAY
32	168	31.2 m	27.6 m	11.6% (11.4% to 11.8%)	0.6 km
64	344	47.2 m	39.1 m	17.2% (16.9% to 17.6%)	2.8 km
96	518	55.4 m	45.6 m	17.6% (17.1% to 18.2%)	5.1 km
128	693	65.7 m	51.8 m	21.1% (20.5% to 21.6%)	9.6 km
192	1044	81.2 m	63.4 m	21.9% (21.2% to 22.7%)	18.5 km
288	1558	111.9 m	75.1 m	32.9% (32.5% to 33.3%)	57.3 km

Two things happen as the lot grows. A driver can see only about 14 m ahead, which in the smallest lot is a large part of the lot and in the largest is a small part of it, so unaided search gets longer. The drive with the app grows too (more cars fill the nearest spots, so the nearest free one is farther away), but more slowly: from 28 m to 75 m with the app, against 31 m to 112 m without it. The share removed rises from 12% to 33%, and because there are more cars as well, the kilometres saved per day rise faster still (Figure 14).

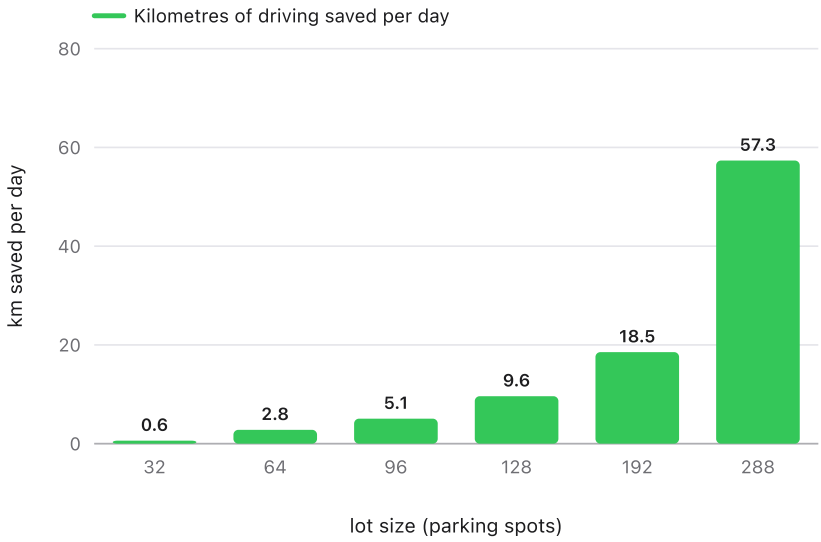


Figure 14. Kilometres of driving removed per day, by lot size.

What that is worth, per 1,000 cars

PER 1,000 CAR VISITS	DRIVING	TIME AT 10 KM/H	CO_2 AT 0.12 KG/KM
Our lot (32 spots)	3.6 km	0.36 h	0.4 kg
128-spot lot	13.9 km	1.39 h	1.7 kg
288-spot lot	36.8 km	3.68 h	4.4 kg

These are modest numbers per car, because a search inside a lot is short. The percentage is the robust result; the absolute savings are small for a small lot and add up for a busy one (the 288-spot lot saves 57 km every day, or about 20,900 km a year). They also leave out everything outside the lot: not queuing at a full entrance, not driving on to another car park, and choosing a spot close to where you are going.

8. How sensitive is the result?

A number from a model is only worth as much as its assumptions. Each chart below changes one assumption and leaves the rest alone (1,000 days per point, the shaded band is the 95% interval).

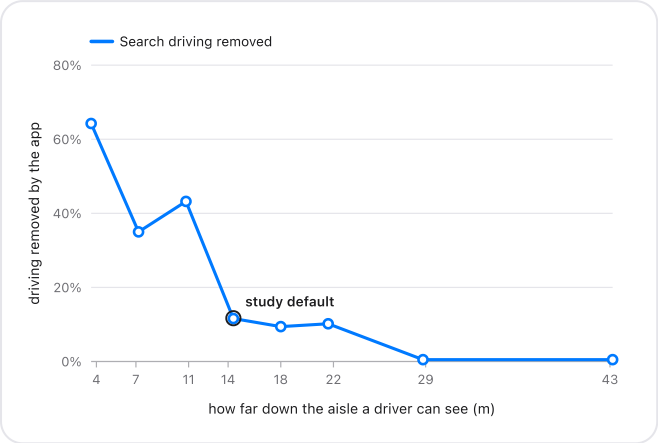


Figure 15. Driver's view. If drivers could see the whole lot, the app would not be needed: at 43 m of view the saving is 0.5%. The shorter the view, the more it saves. The line is not smooth because it follows the geometry of this particular lot: at some viewing distances a whole row of spots comes into view from the entrance.

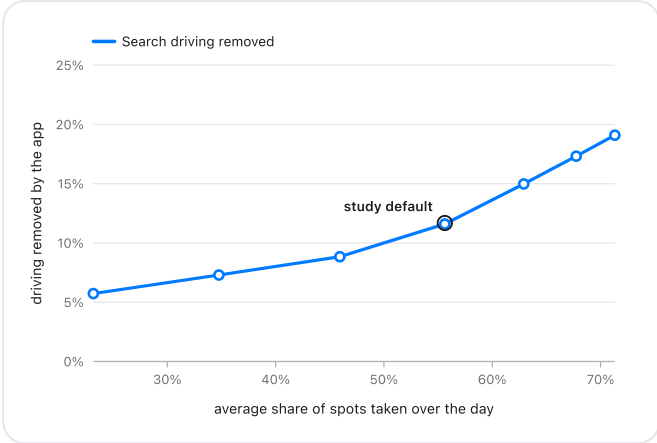


Figure 16. Demand. Each point is a different arrival rate (0.4× to 1.6×), plotted by the average occupancy it produces. Busier lots mean fewer free spots to find.

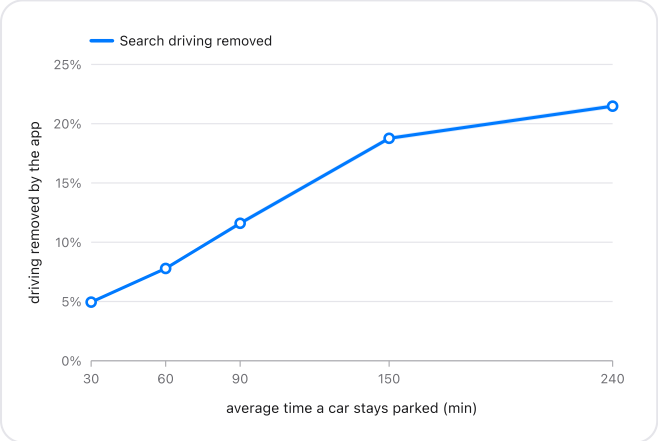


Figure 17. Parking duration. Cars that stay longer keep the lot fuller.

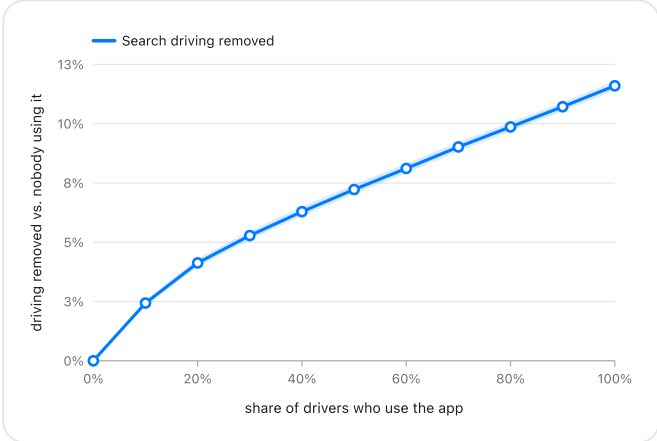


Figure 18. Adoption. The share of drivers who use the app: with one driver in ten on it the lot already gets 2.4% less driving, with half of them 7.2%, with all of them 11.7%. The curve bends: the first adopters gain the most. That is more than the 1.2% you would get if only the app's own users benefited, so the app seems to help drivers who do not use it too, probably because it fills spots from the entrance outwards and leaves fewer gaps for others to search through (we have not isolated the cause).

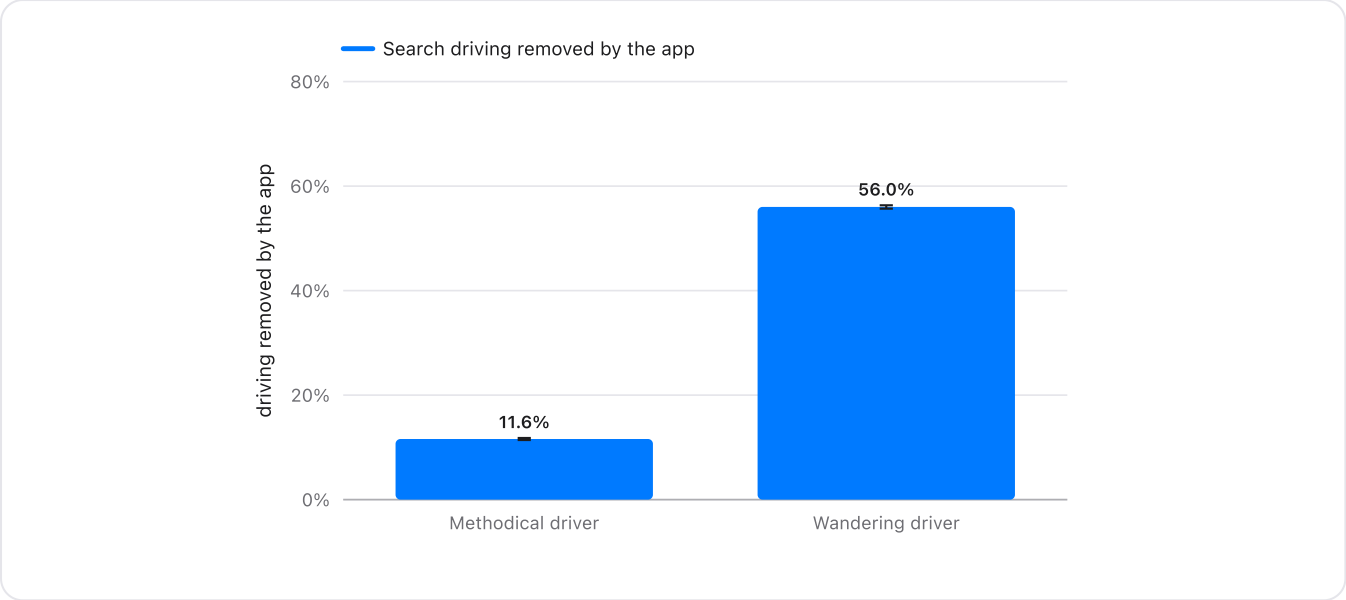


Figure 19. Who the app is compared with. A methodical driver (sees down the aisle and never re-checks a spot): 11.6%. A wandering driver (random turns, first free spot seen): 56.0%, with the unaided drive averaging 62.6 m instead of 31.2 m.

Cell size

The cell size changes the absolute distances but not the percentage, because every distance is multiplied by the same factor:

METRES PER CELL	WITHOUT THE APP	WITH THE APP	SAVED PER DAY	DRIVING REMOVED
2.5 m	21.6 m	19.1 m	0.42 km	11.69%
3 m	25.9 m	22.9 m	0.51 km	11.69%
3.6 m	31.1 m	27.5 m	0.61 km	11.69%
4.5 m	38.9 m	34.4 m	0.76 km	11.69%
5.5 m	47.6 m	42.0 m	0.93 km	11.69%

9. Limits of this study

We would rather say plainly where this model is simple than let the number look better than it is.

- **It is a model, not field data.** Real arrival patterns, real parking durations and the real way people drive in a lot differ from our assumptions. The sensitivity charts show how the answer moves when they do; they do not replace measuring a real lot. The sensors in the project can supply those measurements.
- **The comparison driver is a choice.** Without an app, people do not follow a fixed algorithm. We used a sensible one and a pessimistic one and report both, and the headline is the sensible one. A different assumption about how drivers search changes the answer within the range of Figure 19.
- **The app is assumed to be perfect.** It knows the true state of every spot and drivers follow it. In practice sensors fail, drivers ignore the app, and a spot can be taken between the route being given and the car arriving. The live app handles the last case by re-routing; the study does not charge it for delays.
- **The app picks the spot nearest the entrance.** Drivers who want the spot nearest to a particular building are not modelled; the savings measured here are for getting parked, not for the walk afterwards.
- **Searches do not overlap.** A car's search is instant relative to other cars. The model ignores congestion in the aisles, which would make unaided search worse, so leaving it out is conservative.
- **One layout.** The scaled lots follow the same pattern as our demo lot. A lot with a very different layout (a multi-storey garage, for example) needs its own run.

- **The emission figure is an average.** Slow, stop-and-go driving emits more per km than the average used here, so the CO_2 numbers are conservative.

What we would do next

- Log real arrivals, durations and search distances from the sensors in a real lot and replace the assumed demand with measured demand.
- Add the cost of sensor errors and drivers who ignore the route, and measure how much of the saving survives.
- Model the entrance queue, which a full-lot warning from the app would also shorten.

10. Reproducing every number

```
git clone https://github.com/AvivEli1/SmartParking.git
cd SmartParking && npm install
npm run simulate           # self-checks, then the full study (seed 2026); about a minute or two
npm run simulate:report    # rebuilds this PDF from the results file
```

The same seed gives the same numbers on any machine. The assumptions are in `src/sim/params.ts`; the model is in `src/sim/` (`engine.ts` for the day simulation, `drivers.ts` for the two policies, `dijkstra.ts` for the router). The simulation page of the website runs the same code in the browser, so anyone can change an assumption and see the answer move.

References

- E. W. Dijkstra, "A note on two problems in connexion with graphs", *Numerische Mathematik* 1, 269-271 (1959).
- D. C. Shoup, "Cruising for parking", *Transport Policy* 13(6), 479-486 (2006).

Glossary

TERM	MEANING
Search driving	Distance from the entrance to the spot the car finally parks in.
Paired / common random numbers	Running the identical cars through both policies so only the policy differs.
95% confidence interval	The range that would contain the true value in 95 of 100 repeats of the whole study.
Bootstrap	Estimating that range by re-drawing the simulated days at random many times.
Occupancy	Share of the spots that are taken.
Cell	One square of the lot grid, 3.6 m of driving.